

# Fast uniform-mesh Volterra solvers in **voles**: the algorithm, its provenance, and its verification

Notes accompanying the **fast-uniform-mesh** branch  
(commits 782fb58, 382ec8d, a56349d)

July 2026

## Abstract

These notes document two performance optimizations to the **voles** collocation solvers for Volterra equations, in enough detail that a mathematically literate reader can reproduce them without consulting other literature or the code. Both exploit the same structural fact: on a uniform mesh with a convolution kernel  $K(t - s)$ , every quantity the solvers precompute depends on a pair of mesh intervals only through their *lag*. The centerpiece (Sections 3–4) is a self-contained account of the blocked-FFT algorithm of Hairer, Lubich and Schlichte for accumulating Volterra “history” sums in  $O(Q \log^2 Q)$  operations instead of  $O(Q^2)$ , including a correctness proof of the scheduling we use and a first-principles derivation of the operation count. A second, independent optimization (Section 5) reduces the callable-input solvers’ weight assembly from  $O(M^2)$  quadratures to  $O(M)$ . Headline results: a smooth-kernel function-mode solve at  $M = 320$  went from 1.02 s to 0.013 s; a sampled first-kind solve at  $N = 160,001$  from 19.9 s to 0.26 s; a ten-component system at  $N = 36,001$  from 79.9 s to 2.6 s. All 363 repository tests pass. The floating-point reproducibility policy that governed the implementation, and the verification battery that certifies agreement with the original code at rounding level (worst deviation  $2 \times 10^{-11}$  of solution scale), are described at the end (Sections 6–7).

## Contents

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Provenance: why we went looking</b>                           | <b>2</b> |
| <b>2</b> | <b>Setting and notation</b>                                      | <b>3</b> |
| <b>3</b> | <b>The Hairer–Lubich–Schlichte algorithm</b>                     | <b>3</b> |
| 3.1      | The abstract problem: sequential Toeplitz accumulation . . . . . | 3        |
| 3.2      | What the 1985 paper establishes . . . . .                        | 4        |
| 3.3      | The batching idea, in a picture . . . . .                        | 5        |
| 3.4      | The merge schedule, and its correctness . . . . .                | 5        |
| 3.5      | Executing a merge . . . . .                                      | 6        |
| 3.6      | Operation count . . . . .                                        | 7        |
| 3.7      | Why we implemented this, rather than something else . . . . .    | 8        |
| <b>4</b> | <b>Application to the <b>voles</b> sampled-data solvers</b>      | <b>8</b> |
| 4.1      | The lag blocks of each solver family . . . . .                   | 8        |
| 4.2      | The history engine . . . . .                                     | 9        |

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| <b>5</b> | <b>The companion optimization: one-row weight assembly in function mode</b> | <b>9</b>  |
| 5.1      | The existing quadrature machinery . . . . .                                 | 9         |
| 5.2      | The algorithm . . . . .                                                     | 10        |
| <b>6</b> | <b>Floating-point reproducibility policy</b>                                | <b>10</b> |
| <b>7</b> | <b>Verification methodology</b>                                             | <b>11</b> |
| <b>8</b> | <b>Measured results</b>                                                     | <b>12</b> |

# 1 Provenance: why we went looking

The work originated in a benchmarking study for a paper on collocation methods for weakly singular Volterra integral equations. On the classic Olmstead–Nicholson electrochemistry benchmark [6], the `voles` collocation solvers dominated the classical reference methods (product-integration/Huber, Lubich convolution quadrature) in *accuracy per degree of freedom* — but lost badly in *accuracy per second*. A twenty-line `numpy` implementation of Huber’s method on a graded mesh reached  $10^{-8}$  error in a quarter second, while the collocation solver needed close to a second for  $10^{-6}$ ; and Lubich’s FFT-based convolution quadrature ran three orders of magnitude faster still.

Profiling identified two independent bottlenecks:

1. **Sampled mode** (equally spaced data arrays, compiled core): the per-interval *history sum* — the accumulated contribution of all earlier mesh intervals — costs  $O(Q^2)$  block matrix–vector products over  $Q$  mesh intervals. The constant is small (compiled code), but the quadratic scaling made  $N = 160,000$  samples cost twenty seconds, with each doubling of  $N$  quadrupling the time.
2. **Function mode** (callable kernel and data): essentially 100% of the runtime was Python-side *assembly* of the collocation weight tensor — one adaptive or Gauss–Legendre quadrature per *pair* of mesh intervals, i.e.  $O(M^2)$  quadrature calls, each with interpreter and `scipy` overhead.

The cure for the first bottleneck is a classical algorithm due to Hairer, Lubich and Schlichte [1], which Section 3 develops from scratch; the same idea powers modern fast convolution quadrature [2] and reappears in computer algebra as “relaxed” power-series multiplication [3]. The cure for the second is elementary once the lag structure is noticed (Section 5). In both cases the contribution of this work is not new mathematics but a careful transplantation into an existing, heavily tested solver family — governed by a strict floating-point reproducibility policy (Section 6) that turns out to dictate several design choices.

## 2 Setting and notation

Throughout, the kernel is a convolution,  $K = K(t - s)$ , possibly matrix-valued ( $d \times d$  for a system of  $d$  coupled equations). The solvers treat three equation families:

$$\text{VIE-1: } g(t) = \int_0^t K(t - s) y(s) ds, \quad (1)$$

$$\text{VIE-2: } y(t) = g(t) + \int_0^t K(t - s) y(s) ds, \quad (2)$$

$$\text{VIDE: } y'(t) = a(t) y(t) + g(t) + \int_0^t K(t - s) y(s) ds, \quad y(0) = y_0. \quad (3)$$

**Sampled mode.** The user supplies arrays:  $K$  and  $g$  sampled at  $s = 0, h, 2h, \dots, (N - 1)h$ . With collocation setting (`coll_divs` =  $c$ , `coll_choices` =  $\{c_1 < \dots < c_m\} \subseteq \{1, \dots, c\}$ ), the grid is partitioned into  $Q = (N - 1)/c^2$  *mesh intervals* of  $c^2$  samples each; write  $\Delta = c^2 h$  for the mesh-interval width. On mesh interval  $n$  the unknown is a small vector  $U_n$  (the  $m$  collocation values, or  $dm$  for systems). The solver marches  $n = 0, 1, \dots, Q - 1$ ; at each step it solves a small dense linear system

$$AU_n = b_n \pm G_n, \quad G_n = \sum_{\ell=0}^{n-1} B_{n-\ell} U_\ell, \quad (4)$$

where  $A$  and the local data  $b_n$  are cheap (independent of history) and  $G_n$  is the *history term*. Crucially, the coupling block between intervals  $n$  and  $\ell$  depends only on the lag  $n - \ell$ : it is built from samples of the convolution kernel at fixed offsets from the lag position (explicit formulas in Section 4.1). Evaluating (4) directly for every  $n$  costs  $\sum_n n \approx Q^2/2$  block matrix–vector products — the  $O(Q^2)$  bottleneck.

**Function mode.** The user supplies callables  $K(\cdot)$ ,  $g(\cdot)$  and a mesh  $0 = t_0 < t_1 < \dots < t_M$ . On each interval  $[t_n, t_{n+1}]$  of width  $h_n$  the solution is a polynomial of degree  $p - 1$  determined by collocation at  $p$  nodes  $\tau_{n,i} = t_n + x_i h_n$ ,  $x_i \in (0, 1]$  (VIE-2/VIDE also admit  $x = 0$ ). Weakly singular kernels are declared via a list of singular locations in  $u = t - s$  (e.g.  $u = 0$  for Abel kernels  $K(u) \sim u^{-\alpha}$ ). Here the history blocks are genuine integrals of the kernel against basis polynomials, precomputed into a *weight tensor* by numerical quadrature; the analogous lag structure and its exploitation are the subject of Section 5.

## 3 The Hairer–Lubich–Schlichte algorithm

This section is self-contained. It states the abstract problem, summarizes what the 1985 paper [1] establishes, then develops the algorithm in full: the batching idea, the exact schedule we implement (with a correctness proof), how each batch is executed with an FFT, the operation count, and the reasons this is the right tool for `voles`.

### 3.1 The abstract problem: sequential Toeplitz accumulation

Fix dimensions  $t_{\text{dim}} \times s_{\text{dim}}$  (targets and sources; both equal  $dm$  for VIE-1/VIE-2, and  $(dm) \times (dm + d)$  for VIDE, see Section 4.1) and lag blocks  $B_1, B_2, \dots, B_{Q-1}$ . We must compute, *in order*

$n = 0, 1, \dots, Q - 1$ :

$$G_n = \sum_{\ell=0}^{n-1} B_{n-\ell} u_\ell, \quad \text{then} \quad u_n = \Phi_n(G_n), \quad (5)$$

where  $\Phi_n$  is the local solve of (4) — a black box for our purposes. The defining difficulty is *sequentiality*:  $u_n$  is not known until  $G_n$  has been formed, and  $G_n$  involves all earlier  $u_\ell$ . In signal-processing language this is an *online* (or *relaxed*) convolution: the input sequence is revealed one element at a time, and each output must be delivered before the next input exists.

Two reference points frame the problem:

- **Direct accumulation** costs  $\sim Q^2/2$  block mat-vecs, i.e.  $O(D^2 Q^2)$  scalar operations with  $D = \max(t_{\text{dim}}, s_{\text{dim}})$ .
- **Offline convolution** — if all  $u_\ell$  were known in advance — costs  $O(D^2 Q \log Q)$ : each of the  $D^2$  scalar component sequences is a discrete convolution, computable by FFT.

The algorithm below achieves  $O(D^2 Q \log^2 Q)$  *online*: one extra logarithmic factor is the entire price of sequentiality.

### 3.2 What the 1985 paper establishes

Hairer, Lubich and Schlichte wrote for exactly our situation — discretized Volterra convolution equations — and a reader of the full paper takes away five points:

1. **Scope: any convolution discretization.** Discretize  $y(t) = g(t) + \int_0^t k(t-s) F(s, y(s)) ds$  by *any* quadrature whose weights depend only on the lag — product integration, fractional/convolution quadrature, or (as here) piecewise collocation on a uniform mesh. The discrete equations take the form (5). The algorithm addresses the *evaluation* of the lag sums and is agnostic to which discretization produced them.
2. **Nonlinearity is no obstacle.** Only *already computed* values enter the history; the (possibly nonlinear, possibly implicit) step map  $\Phi_n$  stays local and untouched. This is why the technique drops into an existing solver without changing its per-step logic — the property we rely on.
3. **The core idea: batch past-to-future contributions in square blocks.** Although a single global FFT is impossible, the contribution of a *completed* contiguous block of sources to a comparable-length *future* range of targets is an ordinary (offline) convolution, computable by FFT the moment the block completes. Organizing these blocks in a doubling (power-of-two) pattern covers every (source, target) pair exactly once (Section 3.4).
4. **Cost  $O(Q \log^2 Q)$  time,  $O(Q)$  memory.** Derived in Section 3.6. Memory is one accumulator and one stored source vector per step, plus the lag-block table. (Later work [2] trades exactness of this layout for  $O(\log Q)$  memory when the kernel admits certain integral representations; we do not need that here.)
5. **The result is the same numbers, reordered.** The algorithm evaluates precisely the sums (5), grouping the additions differently and pushing some through FFTs. It is an evaluation strategy, not a new discretization: convergence theory, stability, and outputs are those of the underlying method, up to floating-point reordering (quantified in Sections 6–7). Crossover versus direct accumulation occurs already at a few hundred steps.

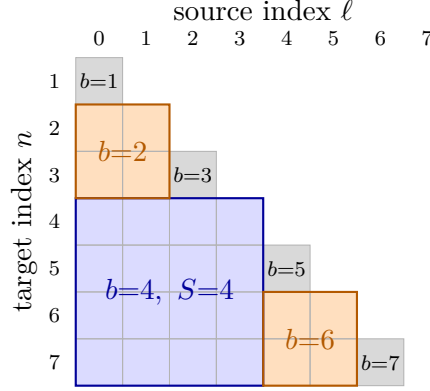


Figure 1: The dependency triangle for  $Q = 8$ , tiled by the merge schedule of Section 3.4. Cell  $(\ell, n)$  is the contribution of source  $u_\ell$  to target  $G_n$ . Each rectangle is one *merge*, performed at boundary  $b$  (after step  $b - 1$ ): sources  $[b - S, b)$ , targets  $[b, b + S)$ ,  $S = 2^{\nu_2(b)}$ . The  $4 \times 4$  block is executed by FFT; small blocks below the cutoff run directly. Every cell lies in exactly one rectangle (Theorem 1).

The remainder of this section is our concrete realization. The scheduling bookkeeping below (one merge per step, sized by a 2-adic valuation) is chosen for ease of implementation and proof; it executes the same doubling decomposition as the original paper.

### 3.3 The batching idea, in a picture

Think of the work as the set of *pairs*  $\{(\ell, n) : 0 \leq \ell < n < Q\}$  — a strictly lower-triangular array in which pair  $(\ell, n)$  means “source  $u_\ell$  contributes  $B_{n-\ell}u_\ell$  to target  $G_n$ .” Direct accumulation visits the triangle cell by cell. The constraint is only this: cell  $(\ell, n)$  may be processed any time *after*  $u_\ell$  exists and *before*  $G_n$  is consumed at step  $n$ .

Any axis-aligned rectangle  $[\ell_1, \ell_2] \times [n_1, n_2]$  lying wholly in the triangle (i.e.  $\ell_2 < n_1$ ) can therefore be processed *as one batch* at any moment between step  $\ell_2$  and step  $n_1$  — and a batch is a plain offline convolution: with sources  $x_u = u_{\ell_1+u}$  and outputs indexed by  $w = n - n_1$ ,

$$\text{batch contribution to } G_{n_1+w} = \sum_u B_{(n_1+w)-(\ell_1+u)} x_u,$$

a convolution of the source block against a contiguous stretch of lag blocks. Batches of size  $S \times S$  replace  $S^2$  cell visits by FFTs of length  $O(S)$  — the entire game is to tile the triangle with rectangles as large as the sequential constraint allows.

Figure 1 shows the tiling for  $Q = 8$ . Near the diagonal the sequential constraint forces small batches (a target one step after a source leaves no room); far from the diagonal, blocks of size comparable to their distance from the diagonal fit comfortably. The doubling pattern makes this precise: blocks of size  $S = 2^k$  sit at distance  $\sim 2^k$  from the diagonal, and each of the  $\log_2 Q$  size classes tiles an  $O(Q)$ -area band of the triangle.

### 3.4 The merge schedule, and its correctness

Write  $\nu_2(b)$  for the 2-adic valuation of the integer  $b \geq 1$ : the exponent of the largest power of two dividing  $b$  (so  $\nu_2(12) = 2$ ,  $\nu_2(7) = 0$ ; in code,  $2^{\nu_2(b)} = b \& (-b)$ ).

**Schedule.** Immediately after step  $n$  produces  $u_n$ , let  $b = n + 1$ . If  $b < Q$ , perform *one* merge: with  $S = 2^{\nu_2(b)}$  and  $E = \min(b + S, Q)$ , add to the accumulators

$$G_{n'} += \sum_{\ell=b-S}^{b-1} B_{n'-\ell} u_\ell, \quad n' \in [b, E). \quad (6)$$

When step  $n'$  is later reached, its accumulator already equals the full history  $G_{n'}$  and is consumed with no further summation.

The schedule is legal — all sources in (6) exist at boundary  $b$ , all targets lie in the future — and it is *complete and non-redundant*:

**Lemma 1** (unique maximal valuation). *Every nonempty integer interval  $(\ell, n]$  contains exactly one element of maximal 2-adic valuation.*

*Proof.* Suppose  $b_1 < b_2$  in  $(\ell, n]$  both have valuation exactly  $v$ . Their difference is a positive multiple of  $2^v$ ; it cannot equal  $2^v$  (then  $b_2 = b_1 + 2^v$  would have valuation  $\geq v+1$ ), so  $b_2 - b_1 \geq 2^{v+1}$  and the midpoint  $b_1 + 2^v \in (b_1, b_2) \subset (\ell, n]$  has valuation  $\geq v+1$ . Hence the maximal valuation in  $(\ell, n]$  cannot be attained twice: an element of strictly larger valuation would lie between the two witnesses, contradicting maximality.  $\square$

**Theorem 1** (exactly-once covering). *For every pair  $0 \leq \ell < n < Q$  there is exactly one boundary  $b \in (\ell, n]$  whose merge (6) transports  $u_\ell$  to  $G_n$ ; namely the unique  $b^* \in (\ell, n]$  of maximal valuation.*

*Proof.* A boundary  $b$  with  $S = 2^{\nu_2(b)}$  covers the pair iff  $b - S \leq \ell < b \leq n < b + S$ .

*Existence.* Let  $b^*$  be the maximal-valuation element of  $(\ell, n]$  (Lemma 1),  $v = \nu_2(b^*)$ ,  $S^* = 2^v$ . Write  $b^* = S^*(2k+1)$ . If  $k = 0$  then  $b^* - S^* = 0 \leq \ell$  trivially; if  $k > 0$  then  $b^* - S^* = 2S^*k$  has valuation  $\geq v+1$ , so it cannot lie in  $(\ell, n]$ , and since  $b^* - S^* < b^* \leq n$  this forces  $b^* - S^* \leq \ell$ . Likewise  $b^* + S^* = 2S^*(k+1)$  has valuation  $\geq v+1$ , so  $b^* + S^* \notin (\ell, n]$ ; since  $b^* + S^* > \ell$  this forces  $b^* + S^* > n$ . Thus  $b^*$  covers the pair.

*Uniqueness.* Suppose  $b' \in (\ell, n]$  with  $S' = 2^{\nu_2(b')}$  also covers the pair,  $b' \neq b^*$ . Then  $\nu_2(b') < \nu_2(b^*)$ , so  $b^*$  is a multiple of  $2S'$ . But the covering conditions place all of  $(\ell, n] \ni b^*$  inside the open interval  $(b' - S', b' + S')$ , whose only multiples of  $2S'$  are its excluded endpoints (writing  $b' = S'(2k'+1)$ :  $b' - S' = 2S'k'$  and  $b' + S' = 2S'(k'+1)$  are *consecutive* multiples of  $2S'$ ). Contradiction.  $\square$

### 3.5 Executing a merge

**Small merges: direct.** Below a cutoff ( $S < 32$  in our implementation) the merge runs as  $S \times (E - b)$  block mat-vecs, exactly like direct accumulation restricted to the rectangle. Over a whole run these small merges touch each pair at most once, so their total cost is bounded by the number of pairs within distance cutoff of the diagonal:  $O(Q \cdot \text{cutoff})$  block operations — negligible, and it spares tiny FFTs whose overhead exceeds their savings.

**Large merges: FFT convolution.** Fix a merge  $(b, S, E)$  and a component pair: target row  $a$ , source column  $c$  of the blocks. Define length- $L$  arrays with  $L = 2S$ :

$$\alpha[u] = u_{b-S+u}[c] \quad (0 \leq u < S, \text{ zero-padded to } L), \quad \beta[v] = B_v[a, c] \quad (1 \leq v \leq 2S-1), \quad \beta[0] = 0,$$

where lags  $v \geq Q$  — possible only when targets were clipped ( $E < b + S$ ) — are zero-filled. For  $0 \leq w < E - b$  the needed sums are a slice of the *linear* convolution:

$$\sum_{\ell=b-S}^{b-1} B_{(b+w)-\ell}[a, c] u_{\ell}[c] = \sum_{u=0}^{S-1} \beta[S+w-u] x[u] = (x * \beta)[S+w].$$

**Lemma 2** (wrap-free window). *The circular (FFT) convolution of length  $L = 2S$  agrees with the linear convolution at positions  $t \in [S, 2S - 1]$ .*

*Proof.* The linear convolution of supports  $[0, S-1]$  and  $[0, 2S-1]$  has support  $[0, 3S-2]$ . Circular aliasing folds index  $t + 2S$  onto index  $t$ ; since  $t + 2S \leq 3S - 2$  forces  $t \leq S - 2 < S$ , no aliased mass lands in  $[S, 2S - 1]$ .  $\square$

So the merge is: one forward FFT per source column ( $s_{\text{dim}}$  of them), one forward FFT per component pair's lag segment ( $t_{\text{dim}} s_{\text{dim}}$ ), pointwise multiply-accumulate into per-row frequency accumulators, and one inverse FFT per target row ( $t_{\text{dim}}$ ); then add the real parts at positions  $S..S + (E - b) - 1$  into the accumulators. All lengths are  $2S$  with  $S$  a power of two, so a plain iterative radix-2 complex FFT [4] suffices — about sixty lines: bit-reversal permutation followed by butterfly passes, split real/imaginary arrays, inverse transform leaving the  $1/L$  scaling to the caller. Scratch buffers are grown on demand and reused across merges.

**Remark 1** (caching variant). *The lag-segment transforms depend only on  $(S, a, c)$ , not on  $b$ , so they could be computed once per size class and cached ( $O(Q \log Q)$  extra memory). We recompute them lazily per merge — simpler,  $O(1)$  extra memory, and measured to be cheap.*

### 3.6 Operation count

*Merge census.* Boundaries with  $\nu_2(b) = k$  are  $b = 2^k(2j + 1) < Q$ , i.e. about  $Q/2^{k+1}$  merges of size  $S = 2^k$ , for  $k = 0, 1, \dots, \lfloor \log_2 Q \rfloor$ . (Sanity check via areas: size- $2^k$  merges cover  $(Q/2^{k+1}) \cdot 4^k = Q 2^{k-1}$  pairs; summing over  $k$  gives  $\sum_k Q 2^{k-1} \approx Q^2/2$ , the whole triangle, consistent with Theorem 1.)

*Cost per merge.* With  $D = \max(t_{\text{dim}}, s_{\text{dim}})$ :  $O(D^2)$  FFTs of length  $2S$  at  $O(S \log S)$  each, plus  $O(D^2 S)$  pointwise work —  $O(D^2 S \log S)$  total.

*Total.*

$$\sum_{k=0}^{\log_2 Q} \underbrace{\frac{Q}{2^{k+1}}}_{\text{\#merges}} \cdot O(D^2 2^k k) = O\left(D^2 Q \sum_{k=0}^{\log_2 Q} k\right) = O(D^2 Q \log^2 Q).$$

An equivalent way to see the  $\log^2$ : each source vector participates in at most one merge per size class (about  $\log_2 Q$  merges over its lifetime — in Figure 1, each column meets one rectangle of each size), and inside a size- $2^k$  merge its per-element FFT share is  $O(k)$ ; summing  $O(k)$  over  $k \leq \log_2 Q$  gives  $O(\log^2 Q)$  work per source, versus  $O(Q)$  for direct accumulation and  $O(\log Q)$  for a single offline FFT. The one extra log factor is the price of sequentiality.

*Memory.* The lag-block table ( $Q t_{\text{dim}} s_{\text{dim}}$  doubles — 1.3 MB for a scalar  $p = 3$  run with  $Q \approx 18,000$ ), the accumulators and stored sources ( $Q(t_{\text{dim}} + s_{\text{dim}})$ ), and  $O(D^2 S_{\text{max}})$  scratch:  $O(D^2 Q)$  overall.

*Empirical confirmation.* After the change, measured times double (rather than quadruple) per doubling of  $N$ : 0.05 s, 0.26 s, 1.32 s at  $N = 40\text{k}$ , 160k, 640k (Section 8) — the residual super-linearity being the  $\log^2$  factor plus the  $O(Q)$  local solves.



### 3.7 Why we implemented this, rather than something else

- **The quadratic wall is the binding constraint.** At the resolutions our applications need ( $N \sim 10^5$ – $10^6$  samples; long-time electrochemical transients),  $O(Q^2)$  costs minutes and each refinement quadruples it. No constant-factor tuning (vectorization, cache blocking) changes that shape; only an asymptotically better summation does.
- **It is an evaluation strategy, not a new method.** Because the algorithm computes the same sums reordered (take-home 5), it drops into each existing driver by replacing “form  $G_n$  by a loop” with “read accumulator; after the step, push  $u_n$ ” — no change to discretization, step maps, boundary handling, or public API, and the entire existing test suite remains the oracle. Alternatives (fast-oblivious quadrature [2], kernel compression) change the representation of the history and would have been a rewrite, not a retrofit.
- **It composes with every solver family at once.** One engine (Section 4.2) serves VIE-1 (both variants), VIE-2 and VIDE, scalar through  $d$ -component systems, because they differ only in their lag-block tables and source vectors.
- **Its floating-point behaviour is controllable.** Sum reordering and FFT rounding perturb results at the  $\varepsilon \log Q$  level — quantifiable, verifiable, and (unlike approximation-based accelerations) carrying no method-error budget to tune. This mattered under the reproducibility policy of Section 6.

## 4 Application to the voles sampled-data solvers

### 4.1 The lag blocks of each solver family

It remains to exhibit each family’s history in the form (5). In all cases the mesh-interval width is  $\Delta = c^2 h$  and  $w_j$  denotes the interpolatory quadrature weights of the node set  $\{c_j/c\}$ .

**VIE-1 and VIE-2.** The scalar blocks are pure kernel samples (systems replace each entry by the corresponding  $d \times d$  kernel block):

$$B_v[i, j] = w_j K([v c^2 + (c_i - c_j) c] h), \quad i, j = 1, \dots, m, \quad v = n - \ell, \quad (7)$$

entering the right-hand side as  $\pm \Delta \sum_{\ell < n} B_{n-\ell} U_\ell$  (sign per family). We fold the prefactor into the stored table,  $\tilde{B}_v = \Delta B_v$ , once at initialization. The continuous (“force-continuous”) VIE-1 variant differs from the discontinuous one only in local terms (a boundary value propagated interval to interval); its history is the same  $B_v$  and needs no special treatment.

**VIDE.** The history couples to both the collocation values  $Y_\ell$  of  $y'$  and the propagated boundary values  $y_\ell^{\text{bnd}} = y(t_\ell)$ :

$$G_n = \sum_{\ell < n} \left[ \Delta^2 C_{n-\ell} Y_\ell + \Delta \kappa_{n-\ell} y_\ell^{\text{bnd}} \right], \quad (8)$$

where the matrix  $C_v$  and vector  $\kappa_v$  are fixed contractions of kernel samples at indices  $vc^2 + (c_i - c_k)c$  against precomputed node tensors (their exact formulas are the existing solver’s and are not modified). This fits (5) with a *rectangular* augmented block: source vector  $s_\ell = [Y_\ell; y_\ell^{\text{bnd}}]$  of dimension  $s_{\text{dim}} = dm + d$  and lag block  $\tilde{B}_v = [\Delta^2 C_v \mid \Delta \kappa_v]$  of shape  $dm \times (dm + d)$ . One subtlety: push the boundary value *at the start* of interval  $\ell$  (that is what (8) uses), even though the push happens after the interval solve, by which time the end value has also been computed.



## 4.2 The history engine

Package Sections 3.4–3.5 as a self-contained object with runtime dimensions  $(t_{\text{dim}}, s_{\text{dim}})$  and three operations:

**initialize** $(Q, t_{\text{dim}}, s_{\text{dim}})$ : allocate the lag table (length  $Q t_{\text{dim}} s_{\text{dim}}$ , filled by the caller with all scalings folded in; lag 0 unused), the accumulator array  $(Q \times t_{\text{dim}}$ , zeroed), and the source store  $(Q \times s_{\text{dim}})$ .

**G** $(n)$ : return accumulator row  $n$  (valid once sources  $0..n-1$  are pushed).

**push** $(s)$ : store  $s$  as the next source; with  $b = (\text{count})$ ,  $S = b \& (-b)$ ,  $E = \min(b + S, Q)$ , run the single merge of (6) — direct if  $S < 32$ , else the FFT form of Section 3.5.

Driver changes are then minimal and identical across families:

replace “ $G \leftarrow \text{loop over } \ell < n$ ” by “ $G \leftarrow \mathbf{G}(n)$ ”; append “**push** $(u_n)$ ” after each interval solve.

We wired it into all six compiled drivers (VIE-1 discontinuous and continuous, VIE-2, VIDE; scalar and system variants) and, because in our applications systems with more than eight components are common, also into the runtime-dimension drivers used for  $d > 8$  (where the local solves go through LAPACK). The compile-time drivers use a thin fixed-dimension wrapper over the same runtime-dimension core — a refactor verified to change no output bit.

## 5 The companion optimization: one-row weight assembly in function mode

The function-mode solvers spend their time not in stepping but in *building* their weight tensor

$$W[n, i, l, k] = \int_{t_l}^{t_{l+1}} K(\tau_{n,i} - s) L_k\left(\frac{s - t_l}{h_l}\right) ds, \quad l < n, \quad (9)$$

( $L_k$ : reference-interval basis polynomials; plus *diagonal* blocks  $l = n$  with upper limit  $\tau_{n,i}$ ), evaluated by numerical quadrature per block —  $O(M^2)$  quadratures in all. On a uniform mesh the substitution  $\sigma = s - t_l$  gives

$$W[n, i, l, k] = \int_0^h K((n - l)h + x_i h - \sigma) L_k(\sigma/h) d\sigma, \quad (10)$$

a function of the lag  $n - l$  alone; the diagonal blocks are independent of  $n$  altogether. This holds for weakly singular kernels too, provided the declared singularity is convolution-style (locations in  $u = t - s$ ): the singular point falls at the same lag-relative position in every row, so even the classification of blocks as smooth or singular is lag-invariant.

### 5.1 The existing quadrature machinery

Reproducing the optimization requires knowing how a block is evaluated:

1. **Classification.** Declared singular  $s$ -locations for  $\tau_{n,i}$  are tested against  $[a, b] = [t_l, t_{l+1}]$  with tolerance  $10^{-12} \max(1, b - a)$ ; interior or endpoint hits send the block to the adaptive path.

---

**Algorithm 1** Uniform-mesh weight-tensor assembly

---

- 1: detect uniformity ( $\max_l h_l - \min_l h_l \leq 10^{-12}\bar{h}$ ) and convolution-style singularity declaration; else run the general  $O(M^2)$  loop unchanged
  - 2: integrate row  $M-1$  with the unchanged per-block machinery, recording masks  $\text{ok}_{\text{off}}[i, \mu]$  (off-diagonal block at lag  $\mu$ , node  $i$  fully GL-accepted) and  $\text{ok}_{\text{diag}}[i]$
  - 3: copy row  $M-1$  band-diagonally into every row (one contiguous slice per row); copy the diagonal template
  - 4:  $\text{cleanlag}[\mu] \leftarrow \bigwedge_i \text{ok}_{\text{off}}[i, \mu]$
  - 5: **for**  $n = 0, \dots, M-2$  **do**
  - 6:     work list  $\{l = n - \mu : \mu \in [1, n], \neg \text{cleanlag}[\mu]\}$  (one **nonzero**, not an  $O(n)$  scan)
  - 7:     **for** each listed block and each node  $i$  with  $\neg \text{ok}_{\text{off}}[i, n-l]$ : re-run classification, GL + two-order check, and fallbacks exactly as the general path would; overwrite
  - 8:     likewise re-run the diagonal block for each  $i$  with  $\neg \text{ok}_{\text{diag}}[i]$
  - 9: **end for**
- 

2. **Smooth path.** Fixed-order Gauss–Legendre at two orders (6 and 8), all basis functions at once; basis entry  $k$  is *accepted* iff  $|v_1 - v_2| \leq 10^{-9} \max(1, |v_2|)$ , storing the order-8 value; rejected entries fall back per-entry to the adaptive path.
3. **Adaptive path.** `scipy.integrate.quad` (QUADPACK [7]), subdivision limit 100, `points=` hints for interior singularities.

## 5.2 The algorithm

Equation (10) suggests integrating a single row that contains every lag — row  $M-1$  — and copying blocks band-diagonally,  $W[n, :, l] \leftarrow W[M-1, :, l + (M-1-n)]$ , with the diagonal template likewise. Our implementation does exactly this for blocks whose row- $(M-1)$  evaluation came entirely from the *deterministic* fixed-order GL path, and *re-evaluates per row*, with byte-identical calls, every block that involved adaptive quadrature (singularity-classified blocks and two-order-check fallbacks). The reason for treating the two classes differently is a floating-point reproducibility requirement, discussed in Section 6; here we record the algorithm:

Two implementation notes. (i) *Subset determinism*: when only a subset of a block’s nodes is recomputed, the vectorized GL evaluation over the subset must reproduce, per node, the full-set values bit for bit; this holds because the per-node arithmetic (kernel samples per node–abscissa pair, contraction per node) is independent, as in the existing `einsum` formulation. (ii) *Cost*:  $O(M)$  quadratures for the integrated row, plus per-row repairs — for an Abel kernel the  $p$  diagonal blocks and a few near-diagonal fallback lags per row, still  $O(M)$  in total — plus  $O(M^2 p^2)$  pure memory copies. Smooth kernels, having no adaptive blocks, gain the most (measured  $\sim 80\times$  at  $M = 320$ ); Abel kernels gain  $3\text{--}26\times$  and now scale linearly. Non-uniform (graded) meshes and callable singularity locators take the untouched general path, verified bit-identical.

## 6 Floating-point reproducibility policy

The acceptance criterion for this work, fixed during review of an early draft of the assembly optimization, was: *optimized code must agree with the original to numerical precision* — rounding-level deviations are acceptable, tolerance-level deviations are not. Three classes of deviation arise:

1. **Reuse of deterministic quadrature (permitted).** Fixed-order GL is a pure function of interval endpoints and integrand samples. Evaluating an equal-lag block at a shifted absolute position perturbs each abscissa and each  $\tau - s$  argument by a few ulps of the absolute times; equal-lag GL values agree to  $\sim 10^{-15}$  relative. Copying them is as good as recomputing.
2. **Reuse of adaptive quadrature (forbidden).** QUADPACK’s subdivision and extrapolation decisions branch on error estimates formed from absolute coordinates, and its result is guaranteed only to its own tolerance ( $\approx 1.5 \times 10^{-8}$ ). Empirically, equal-lag adaptive blocks evaluated in different rows differ by  $10^{-10}$ – $10^{-9}$  — *including in the original code*, which silently assigns slightly different values to mathematically identical blocks. Reusing one row’s value stays within that pre-existing scatter but is not rounding-level identical to the per-row result; under the criterion we therefore recompute all adaptive blocks per row (making them bit-identical), at the price of keeping an  $O(M)$  per-row quadrature cost on singular kernels. The rejected “full reuse” variant is a further  $\sim 8\times$  on Abel kernels and is worth revisiting if the criterion is ever relaxed. A residual measure-zero caveat: a two-order acceptance check comparing values that differ across rows by  $10^{-15}$  against a sharp threshold could in principle flip for some row; the consequence is bounded by the GL-versus-adaptive gap ( $\lesssim 10^{-9}$ ) on that single block.
3. **Summation restructuring (permitted).** The history engine reorders additions and routes some through FFTs. Deviations are rounding-level,  $O(\varepsilon \log Q)$  relative to the magnitudes in the sums, mildly amplified by problem conditioning (first-kind equations show the largest values) — the same class as compiling with different SIMD settings, and unavoidable in *any* fast summation. Measured worst case over the entire verification battery:  $2 \times 10^{-11}$  of solution scale; typical  $10^{-15}$ – $10^{-13}$ .

## 7 Verification methodology

Reproducing the verification matters as much as the algorithms.

**A/B protocol.** Build the unmodified reference and the optimized branch as two installations; run *identical* scripts against both; compare saved outputs. The certificate sought is agreement with the old code, not correctness of the discretization — the latter is the existing test suite’s job (all 363 tests pass on the optimized build,  $1.8\times$  faster).

**Coverage.** (i) Every problem spec of the repository test suite (20 specs across the three families, including the weakly singular ones), each run through both the sampled and function solvers; (ii) each at the suite’s own size *and* at a larger size that actually exercises the FFT merges and the one-row assembly (the suite’s  $Q = 10$  never reaches the merge cutoff); (iii) vector cases ( $d = 2$ ) and a  $d = 10$  battery for the runtime-dimension drivers — about 100 A/B cases in all.

**Two traps, so the reader avoids them.**

1. **Growing-resolvent problems.** Enlarging a test problem by extending its horizon at fixed step is wrong. Several suite problems have exponentially growing resolvents (a second-kind equation with  $K = 2$  grows like  $e^{2t}$ ); by  $T \approx 46$  the numerical solution in *both* builds is

dominated by rounding-seeded growing modes ( $10^6$ – $10^{24}$  against true solutions of  $O(1)$ – $O(10^3)$ ), and the two builds then differ at  $O(1)$  relative — chaotic amplification of last-bit noise, not a bug. Keep the horizon; refine the step to reach large  $Q$ . After this correction the worst case dropped from  $O(1)$  to  $2 \times 10^{-11}$ .

2. **Zero crossings.** Element-wise relative comparison fails spuriously where solutions cross zero: rounding noise has absolute size set by the magnitudes in the *computation* (history sums involve the whole solution), not by the local output value. Use a per-problem scale-aware floor, `atol = tol · max |y|`: with `rtol = tol = 10-10` all finite cases pass; with `numpy` defaults all 100 cases pass.

Two deliberately borderline extra cases (continuous VIE-1 with nodes  $\{0.5, 1.0\}$ , amplification exactly  $\rho = 1$ ) overflow to `inf/NaN` on long runs — identically in both builds (same positions, finite prefixes agreeing to  $\sim 10^{-167}$  relative); they are excluded from finite statistics.

**Bitwise identities.** Verified exactly: (a) graded (non-uniform) meshes produce bit-identical weight tensors (the fast path must not engage); (b) the runtime-dimension refactor of the history engine changed no output bit over the whole battery.

**Recommended test order for a reimplementaion.** (a) history engine versus direct summation on *random* lag tables — a pure unit test with no solver attached; (b) the A/B battery above; (c) the full existing test suite; (d) timing curves — confirm  $4\times$  per doubling becomes  $\approx 2\times$ .

## 8 Measured results

Apple-silicon laptop, July 2026; collocation order  $p = 3$  (`coll_divs=3`, `coll_choices=[1,2,3]`) unless noted; “old” is upstream `main` built identically (extrapolated entries marked).

| case                                   | old          | new     | speedup          | scaling change                     |
|----------------------------------------|--------------|---------|------------------|------------------------------------|
| sampled VIE-1, $N = 160,001$           | 19.9 s       | 0.26 s  | $77\times$       | $O(Q^2) \rightarrow O(Q \log^2 Q)$ |
| sampled VIE-1, $N = 640,001$           | $\sim 320$ s | 1.32 s  | $\sim 240\times$ |                                    |
| sampled VIE-2, $d = 10$ , $N = 36,001$ | 79.9 s       | 2.56 s  | $31\times$       | runtime- $d$ drivers               |
| function, smooth kernel, $M = 320$     | 1.02 s       | 0.013 s | $\sim 80\times$  | $O(M^2) \rightarrow O(M)$ quads    |
| function, Abel kernel, $M = 320$       | 3.31 s       | 0.98 s  | $3.4\times$      |                                    |
| function, Abel, $M = 2560$             | $\sim 210$ s | 8.0 s   | $\sim 26\times$  |                                    |
| repository test suite (363 tests)      | 35.5 s       | 19.4 s  | $1.8\times$      |                                    |

Per-test suite timings and full difference distributions are in the companion figures (`plot_speedups.png`, `plot_differences.png`).

### Known limitations / next steps

Graded (non-uniform) meshes take the original paths by design; the natural extension is a graded-start/uniform-tail hybrid. The function-mode dense weight tensor still costs  $O(M^2 p^2)$  memory; a lag-indexed stepping entry point would cut it to  $O(M p^2)$ . Function mode has no  $d > 8$  capability at all (it errors rather than falling back); given that large systems matter in our applications, that is a missing feature, not a slow path. And the “full reuse” assembly variant of Section 6 remains available should the reproducibility criterion ever be relaxed.

## References

- [1] E. Hairer, C. Lubich, and M. Schlichte, *Fast numerical solution of nonlinear Volterra convolution equations*, SIAM J. Sci. Stat. Comput. **6** (1985) 532–541.
- [2] A. Schädle, M. López-Fernández, and C. Lubich, *Fast and oblivious convolution quadrature*, SIAM J. Sci. Comput. **28** (2006) 421–438.
- [3] J. van der Hoeven, *Relax, but don't be too lazy*, J. Symbolic Comput. **34** (2002) 479–542.
- [4] J. W. Cooley and J. W. Tukey, *An algorithm for the machine calculation of complex Fourier series*, Math. Comp. **19** (1965) 297–301.
- [5] H. Brunner, *Collocation Methods for Volterra Integral and Related Functional Differential Equations*, Cambridge University Press, 2004.
- [6] M. L. Olmstead and R. S. Nicholson, *A method based on polynomial approximations for numerical solution of Volterra integral equations*, J. Electroanal. Chem. **16** (1968) 145–151.
- [7] R. Piessens, E. de Doncker-Kapenga, C. Überhuber, and D. Kahaner, *QUADPACK: A Subroutine Package for Automatic Integration*, Springer, 1983.